# Week 1: Everything is a Vector

## Consulting, Storage, and Embeddings

Instructor / Mentor

Kiki Research Training Program

Week 1

# Contents

1. Advisory: Navigating College Admissions
2. Today's Core Concept
3. The Box Game & Representation
4. Hardware: Binary vs. Decimal
5. LLMs and Embeddings
6. Data Types and Mathematical Speed
7. Vibe Coding Tasks
8. Homework
9. Conclusion: Closing the Loop

# Advisory: Navigating College Admissions

# The Admission Anxiety

### The "Checklist" Mindset

Many students in a new environment want a simple checklist: *"Tell me exactly what boxes to check to get into a top STEM university."*

Before rushing to hire expensive professional counselors, **hold on a little bit**.
Your first step should be talking to current college students (specifically Asian/Chinese students who recently navigated US high schools and applied for STEM majors).

# Why Talk to Peers Before Counselors?

- **High Sunk Costs:** If you engage with the wrong counselor, you waste not only money but *time*—which you cannot get back.
- **Misaligned Goals:** Many agencies prioritize "closing the deal" over your actual long-term research or admission results.
- **Resource Allocation:** Building a real "storyline" requires money (summer schools, research projects, competitions). Save your budget for the actual *work*, not just the advice.

# Action Plan: The 30-Minute Interview

Even if you hire a professional later, talking to peers helps you know *how to choose the right one*.

**Task: Prepare a Question List**

To fully leverage a half-hour talk with a current college student, you must prepare. Let's think about the most critical questions to ask them regarding their high school timeline, real project impacts, and mistakes to avoid.

# Today's Core Concept

# Question of the Day

## "Everything is a Vector."

**Is this sentence correct?**

*Answer:* Not fully. It's a great analogy, but scientifically, it has limitations. By the end of today's class, you will understand exactly *why* it's a good description, but not the ultimate truth.

# The Box Game & Representation

# The Box Game: Rules

Imagine we have physical boxes.

- Each box can store **only one integer number** (0 to 9).
- No letters, no symbols. Just 10 choices per box.

**Scenario:** We need to store the concepts of `Apple`, `Orange`, and `Watermelon`. If we only use *one* box per item, we could assign: Apple = 1, Orange = 2, Watermelon = 3.

# Scaling Up the Box Game

What happens when we need to represent 100 different fruits?

- One box (0-9) is mathematically impossible to represent >10 items without overlap.
- **Solution:** We need a *sequence* of boxes.
- **Optimization:** We can assign properties to boxes.
  - Box 1 = Color (1:Red, 2:Orange, 3:Green...)
  - Box 2 = Size (1:Small, ..., 9:Huge)

# Takeaways from the Box Game

## 1. Representation Efficiency

We can use numbers to represent *anything* (fruits, aliens, words) as long as the mapping is reversible. The efficiency varies depending on how many boxes you use vs. how many items exist.

## 2. The Algorithm Constraint

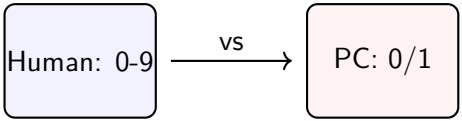
If I give you infinite boxes and infinite time, you can represent and calculate anything. But real life requires achieving the goal within a **time budget** and **limited memory**. That is the true purpose of an algorithm.

# Hardware: Binary vs. Decimal

# Physical Boxes in Computers

Our brains use 10 symbols (0-9) easily. But physical computers cannot.

- A computer's box can only hold **two symbols**: 0 or 1.
- This is dictated by hardware constraints (voltage levels, magnetic charges).



# Reducing the Symbols Game

Let's simulate how a computer feels.

- Imagine you don't like the number '4'. We remove it. Now we only have 9 symbols to represent the world.
- Pick another number you hate. Now we have 8.
- Keep reducing until we only have **two symbols left** (e.g., 0 and 1).

**The Consequence:** To represent the same amount of items with fewer symbols, you *must* use longer sequences (more boxes).

# How Computers Physically Store 0 and 1

At the physical layer, computers use states of matter to represent binary bits:

| Hardware | Physical Mechanism |
|---|---|
| **Memory (DRAM)** | Uses **Capacitors**. Charged = 1, Empty = 0. Requires constant refreshing (hundreds of times per second) due to leakage. |
| **SSD (Flash)** | Uses **Floating Gate Transistors**. Traps electrons in an insulated gate using high voltage. Reads by checking the threshold voltage. |
| **HDD (Mechanical)** | Uses **Magnetic orientation**. Magnetic north up or down. Read by a Giant Magnetoresistance (GMR) sensor. |

# LLMs and Embeddings

# Human Language vs. Computer Language

Our brains hold a massive vocabulary. How many symbols (words/tokens) does a language actually have?

Large Language Model (LLM) developers answer this with **Vocabulary Size**.

Table: Vocabulary and Embedding Sizes of Open Source LLMs

| **Model Series** | **Parameters** | **Hidden Size ($d_{model}$)** | **Vocab Size (Tokens)** |
|---|---|---|---|
| Llama 3.1 | 8B / 70B / 405B | 4096 / 8192 / 16384 | 128,256 |
| Qwen 2.5 | 7B / 72B | 3584 / 8192 | 151,936 |
| Gemma 2 | 9B / 27B | 3584 / 4608 | 256,000 |

# The Tokenizer and Embedding Bridge

How does a human sentence become 0s and 1s?

1. **Tokenization:** The sentence is chopped into "Tokens" (like pieces of words).
2. **Embedding:** Each Token is mapped to a long list of numbers (a Vector). The length is the Hidden Size (e.g., 4096 numbers for Llama 3.1 8B).
3. **Matrix Stacking:** These vectors stack together to form an **Embedding Matrix**.

**The Catch**

The numbers in the embedding matrix are not 0 or 1! They are decimals (Floats).

# Data Types and Mathematical Speed

# Data Types: How many boxes do they take?

Those "Float" numbers must eventually be stored as bits (boxes).

| Language | Data Type | Box Cost (Bits) |
|---|---|---|
| **Python** | `int` | Variable (28+ bytes, expands automatically) |
| | `float` | 64 bits (IEEE 754 Double Precision) |
| | `str` | 8, 16, or 32 bits per character (ASCII vs Unicode) |
| **MATLAB** | `double` | 64 bits |
| | `char` / `string` | 16 bits per char (UTF-16) / Plus container overhead |
| **R** | `numeric` | 64 bits |
| | `character` | 64-bit pointer to an 8-bit/char global pool |

# Why are Matrix Operations so Fast?

Python and MATLAB are interpreted languages, yet their matrix math is nearly as fast as C or Fortran. Why?

- **Bottom-Level Libraries:** NumPy and MATLAB link to BLAS/LAPACK (e.g., Intel MKL). They split huge matrices into chunks that fit perfectly into CPU L1/L2 caches.
- **SIMD (Single Instruction, Multiple Data):** Utilizing AVX-512 instructions, a CPU can calculate 8 to 16 floating-point additions in *one single clock cycle*.
- **Memory Layout:** NumPy uses C-style (Row-major), MATLAB uses Fortran-style (Column-major) block storage to prevent cache misses.

# Vibe Coding Tasks

# Vibe Coding Task 1: Peeking into the Matrix

**Goal:** See the bottom-level 0s and 1s of different data types in Python.

## Prompting Hints for your AI

1. Ask the AI how to use Python in Google Colab.
2. Ask the AI: "Write a Python script to print the binary representation (0s and 1s) of the integer 4, the float 3.14, and the string 'A'."
3. Notice how complex the float's binary representation is compared to the integer. (Hint: Ask the AI to explain the IEEE 754 standard).

# Vibe Coding Task 2: Generating Embeddings

**Goal:** Convert a human sentence into an Embedding Matrix using a real AI model.

## Prompting Hints for your AI

1. Ask the AI how to set up the `openai` Python package and authenticate an API key.
2. Ask the AI to write code that calls the `text-embedding-3-small` model for the sentence: "Everything is a vector."
3. Ask the AI to print the length (dimension) of the resulting vector, and print the first 5 numbers of that vector.

# Homework

# Homework Assignment 1

## API & Euclidean Distance

Use Python to call the OpenAI API (or another embedding API) by yourself.

- Obtain the embedding vectors for three separate words: "apple", "orange", and "watermelon".
- Use Python to calculate the Euclidean distance between Apple & Orange, and Apple & Watermelon.
- **Observe:** Which distance is smaller? Why does the model think they are closer in high-dimensional space?

# Homework Assignment 2

## Survey and Analysis Report (1 Page)

Write a short research report answering the following question:

**"How many '0/1 boxes' (bits) can we theoretically create based on the remaining silicon mine resources on Earth?"**

*Hint:* You will need to research global silicon reserves, the average mass of silicon required to manufacture a single transistor, and extrapolate. Think about the physical limits of computation!

# Conclusion: Closing the Loop

# Closing the Loop: Vector vs. Array

Let's answer our opening question: **Why is "Everything is a Vector" not fully correct?**

### The Vector (Math/Physics)

- A physical/mathematical quantity with magnitude and direction.
- Exists in a continuous, abstract mathematical space.
- Has infinite theoretical precision.

### The Array (Computer Science)

- A data type: an organized group of numbers handled by your CPU.
- Exists in physical memory (a limited number of physical boxes).
- Constrained by floating-point approximations.

### The Physical Reality

In AI and programming, what we call a "vector" is actually just an **array**. We use a limited sequence of 0/1 boxes (the array) to *approximate* a true mathematical vector. Therefore, "Everything is an Array" is the true engineering reality!

# Thank you!

Remember: "Everything is a vector... but every vector is bound by physical boxes."